Production Support Copilot

Beginner-friendly Hinglish Handbook

RAG, embeddings, vector search, LLM integration, Kafka, tools, approvals, security, evaluation and system design - explained through one runnable .NET 8 portfolio project.

LEARNING PATH	BUILD -> MEASURE -> SECURE -> EXPLAIN

Generated from the repository's canonical Markdown knowledge base.

How to use this handbook

First RAG chapters padho, then Swagger demo run karo, phir class catalog ke saath source files open karo. Interview chapter ko loud voice mein explain karo. Numbers marked as assumptions are capacity exercises, not measured production claims.

Production Support Copilot - Hinglish Learning Journal

Ye file tumhare liye personal notebook hai. Agar kal conversation context lost ho jaye, sirf ye repo docs padhkar tum samajh paoge ki project kya hai, kyu banaya, kaise bana, aur interview/demo me kaise explain karna hai.

1. Project ka goal

Hum ek polished portfolio product bana rahe hain: **Production Support Copilot for Event-Driven .NET Services**.

Simple words me:

- Support engineer question poochta hai.
- System authorized tenant ke documents/runbooks/tickets search karta hai.
- Relevant chunks retrieve karta hai.
- LLM answer banata hai, but citations ke saath.
- Agar current system state chahiye, to safe mock tools call karta hai.
- Agar action dangerous/consequential hai, jaise DLQ replay, to pehle human approval maangta hai.

Recruiter/interviewer ke liye yeh strong hai kyunki isme sirf chatbot tutorial nahi hai. Isme production concerns hain: auth, tenant isolation, Kafka, idempotency, DLQ, vector search, evaluation, observability, Docker and architecture tradeoffs.

2. Tech stack

- .NET 8 ASP.NET Core API
- C# application/domain/infrastructure layering
- PostgreSQL
- pgvector
- EF Core migrations
- Kafka-compatible Redpanda
- Python FastAPI embedding service
- SentenceTransformers MiniLM model
- Optional Ollama local LLM
- Docker Compose
- xUnit tests
- OpenTelemetry hooks

Important design choice:

> Python sirf waha use kiya jahan uska AI ecosystem useful hai: embeddings and evaluation. Product API/orchestration .NET me hai, kyunki tumhara core strength senior .NET engineering hai.

3. High-level architecture

Flow:

1. User JWT token ke saath question bhejta hai.

2. API tenant claim verify karta hai.
3. Prompt-injection guard suspicious input block karta hai.
4. Retriever tenant-specific chunks search karta hai.
5. Lexical and vector results RRF se combine hote hain.
6. Evidence ko untrusted document boundary me wrap kiya jata hai.
7. LLM provider answer + cited chunk IDs return karta hai.
8. Service citation IDs validate karta hai.
9. Final response answer, citations, model, tokens and latency ke saath return hota hai.

4. RAG kya hai?

RAG ka full form hai Retrieval Augmented Generation.

- Normal LLM apni training memory se answer banata hai.
- RAG pehle tumhare documents se relevant context laata hai.
- Fir LLM ko bolta hai: sirf is evidence ke basis par answer do.

Interview line:

> RAG hallucination ko zero nahi karta, but answer ko evidence-grounded banata hai. Real safety citation validation, tenant filtering, abstention and evaluation se aati hai.

5. Embeddings kya hain?

Embedding text ka numeric representation hota hai.

Example:

- "consumer lag is high"
- "Kafka consumer is falling behind"

Words different hain, meaning similar hai. Embedding vectors in dono ko vector space me paas la sakte hain.

Project me:

- Python service MiniLM model use kar sakti hai.
- Default local mode deterministic embedding use karta hai, taaki paid key/model dependency ke bina demo run ho.
- Document chunks ka vector `vector(384)` column me store hota hai.

6. Vector database kya hai?

Vector database similarity search karta hai. Hum separate vector DB nahi laaye. PostgreSQL plus pgvector use kiya.

Reason:

- Less infra early stage.
- Metadata, tenant auth and vectors same database transaction boundary me.
- Portfolio ke liye simpler and production-minded.

7. Fine-tuning vs RAG

Fine-tuning model behavior/style/pattern ko train karta hai. RAG fresh factual knowledge ke liye better hai. Runbook/ticket update hota hai to index update fast.

Interview line:

> I would not fine-tune first for changing runbooks. I would use RAG, measure retrieval quality, then consider fine-tuning only for stable behavior patterns.

8. Solution projects

Domain

- `Entity`: common ID and timestamps.
- `Tenant`: tenant isolation root.
- `SupportDocument`: source text, hash, version, classification.
- `DocumentChunk`: text chunk, offsets, token estimate, 384D embedding.
- `IngestionJob`: event ID, attempts, status, error.
- `ApprovalRequest`: pending/approved/rejected/expired decision state.

Application

- `CopilotQueryRequest`, `CopilotQueryResponse`
- `IDocumentChunker`
- `IDocumentIngestionService`
- `IHybridRetriever`
- `IEmbeddingProvider`
- `ILlmProvider`
- `ISupportTool`
- `ISupportWorkflowService`
- `IAiUsageRecorder`

Infrastructure

- EF Core DbContext
- PostgreSQL/pgvector retrieval
- Kafka publisher/worker
- Python embedding HTTP adapter
- Ollama LLM adapter
- DB seeding
- Approval persistence

API

- Swagger
- JWT auth
- rate limit
- middleware

- endpoints
- OpenTelemetry
- DI registrations

9. EF Core and PostgreSQL kaise injected hai?

`DependencyInjection.cs` me Infrastructure layer service registration karta hai:

```
services.AddDbContext<SupportCopilotDbContext>(options =>
    options.UseNpgsql(connection, npgsql => npgsql.UseVector()));
```

Meaning:

- API config se connection string leta hai.
- EF Core ko Npgsql provider ke saath configure karta hai.
- pgvector extension enable karta hai.
- Jahan `SupportCopilotDbContext` constructor me needed hai, DI automatically object provide karta hai.

Tables:

- `tenants`
- `support_documents`
- `document_chunks`
- `ingestion_jobs`
- `approval_requests`

10. Document ingestion

Ingestion steps:

1. `eventId` se duplicate check.
2. Tenant find/create.
3. `IngestionJob` create and start.
4. Document insert/update.
5. Old chunks remove if document updated.
6. Content chunks me split.
7. Each chunk embedding generate.
8. Chunks persist.
9. Job complete.
10. Failure par job failed/dead-lettered.

Why idempotency?

Kafka at-least-once delivery me same event dubara aa sakta hai. Event ID unique rakha, to duplicate event duplicate chunks nahi banata.

11. Retrieval

Retrieval two signals combine karta hai:

- Lexical: exact word match, IDs/error names ke liye strong.

- Semantic: embedding similarity, meaning match karta hai.
- Hybrid + RRF: lexical and semantic ranks combine karta hai.

12. LLM provider abstraction

Implementations:

- `ExtractiveDemoLlmProvider`: free, deterministic, no paid key.
- `OllamaLlmProvider`: local model structured JSON output.
- `ResilientLlmProvider`: primary fail ho to fallback.

Why useful?

- Vendor lock-in kam.
- Test stable.
- Demo paid API ke bina run hota hai.
- Production me OpenAI/Azure OpenAI/provider add karna simple.

13. Prompt-injection defense

Defense layers:

- user input guard
- documents wrapped as untrusted evidence
- system instruction says documents are not instructions
- tenant filtering before retrieval
- cited chunk IDs allow-list validation
- insufficient evidence par abstain

Important line:

> Prompt alone security boundary nahi hota. Authorization retrieval/database layer par enforce honi chahiye.

14. Tools and agent workflow

Tools:

- `service-health`: synthetic current health
- `consumer-lag`: synthetic Kafka lag
- `booking-status`: synthetic booking status lookup
- `replay-dead-letter`: consequential action, approval required

Why explicit state machine?

Autonomous agent flexible hota hai but harder to audit. Production support me explicit states safer hain.

15. Kafka and DLQ

Kafka path:

- API publishes document ingestion command.
- Worker consumes.

- Manual commit after successful processing.
- Failure diagnostic envelope DLQ topic me ja sakta hai.

16. Observability

Implemented:

- Correlation ID middleware
- Request telemetry middleware
- p50/p95 latency snapshot
- AI usage token/cost tracking
- OpenTelemetry ASP.NET/HTTP hooks
- `ProductionSupportCopilot` ActivitySource for RAG spans
- dashboard endpoint

17. Docker run

```
docker compose up --build
# If docker compose plugin available nahi:
docker-compose -f compose.yaml up --build
```

Light mode:

```
docker compose up -d postgres
# Or:
docker-compose -f compose.yaml up -d postgres
dotnet run --project src/ProductionSupportCopilot.Api
```

18. Swagger demo script

Open:

```
http://localhost:5051/swagger
```

Steps:

1. `POST /api/v1/auth/demo-token`
2. Copy access token.
3. Swagger Authorize me `Bearer <token>` paste.
4. `POST /api/v1/retrieval/hybrid-search`
5. `POST /api/v1/copilot/query`
6. `POST /api/v1/copilot/query/stream`
7. `POST /api/v1/workflows/run`
8. `GET /api/v1/tools`
9. DLQ replay tool call.
10. Approval decision endpoint.
11. Prompt injection test.
12. Tenant mismatch test.

19. PR #3 repair log - 2026-07-09

Problem:

- PR branch old `master` se bana tha.
- Current `origin/master` me document model/chunking changes already aa chuke the.
- Merge conflicts aa rahe the in `Program.cs`, `DbContext`, `compose.yaml`, docs and tests.
- Generated junk files PR me accidentally tracked the: `__pycache__`, `*.pyc`, `tmp/pdfs/*.png`.

Decision:

- Current `Domain.Knowledge` model ko source of truth banaya.
- Old duplicate `Domain.Documents` and `Domain.Tenants` tutorial model remove kiya.
- API composition root clean kiya.
- EF model/current migrations align kiya.
- Generated temp artifacts untrack/remove kiye.

Verification target:

```
dotnet build ProductionSupportCopilot.sln
dotnet test ProductionSupportCopilot.sln
docker-compose -f compose.yaml config --quiet
```

20. Interview 90-second pitch

"I built a multi-tenant production support copilot for event-driven .NET services. Documents enter through an idempotent ingestion path, can be processed asynchronously through Kafka, and are chunked, embedded and stored in PostgreSQL with pgvector. Serving applies JWT tenant checks before retrieval, combines lexical and vector search with RRF, and sends only authorized untrusted evidence to an LLM provider abstraction. Answers are validated against cited chunk IDs and safely abstain when evidence is missing. The system also supports controlled tool calls for synthetic live state, while consequential operations like DLQ replay create human approval requests instead of executing directly. I added prompt-injection defenses, rate limits, correlation IDs, OpenTelemetry hooks, token/cost dashboard, tests, Docker Compose, evaluation data and architecture docs."

Class and Service Catalog

Related: 01 Architecture Map, 02 RAG From Zero, 05 Runbook and Demo.

Domain classes

Class	Kya rakhta hai	Important behavior/invariant
`Entity`	ID and timestamps	`MarkUpdated` state change time refresh karta hai
`Tenant`	stable key and display name	empty values reject; documents ownership root
`SupportDocument`	source text, hash, version, classification	replacement hash/version update; tenant+external ID unique
`DocumentChunk`	ordinal, offsets, text, token estimate, vector	exactly 384 dimensions enforce
`IngestionJob`	event ID, attempts, status, error	duplicate event tracked; state transitions explicit
`ApprovalRequest`	action, arguments, requester, expiry/decision	one decision only; expiry check

Application contracts and services

Class/interface	Role	Why separate?
`ChunkDraft`	chunker output value	EF entity create hone se pehle pure data
`IngestDocumentCommand`	ingestion message contract	HTTP/Kafka same contract use karte
`IDocumentChunker`	chunk algorithm boundary	alternative token-aware chunker testable
`DeterministicDocumentChunker`	paragraph/sentence boundary chunks	repeatable indexing/evaluation
`IDocumentIngestionService`	end-to-end indexing use case	API/consumer implementation details se free
`IIngestionEventPublisher`	async boundary	direct learning mode vs Kafka mode
`RetrievalQuery`	tenant/query/topK/filter	authorization-aware search input
`RetrievedPassage`	ranked evidence	raw component + combined scores observable
`IEmbeddingProvider`	text -> float vector	model/service swappable
`IHybridRetriever`	query -> passages	Postgres/search implementation swappable
`IPromptInjectionGuard`	inspect and mark trust	safety policy centralized
`PromptInjectionGuard`	basic phrase checks + boundary tags	defense-in-depth first layer
`LlmRequest`	question + evidence + system policy	clear model contract
`LlmAnswer`	answer + cited IDs + tokens	structured validation and metrics
`ILlmProvider`	model abstraction	vendor/local fallback
`ExtractiveDemoLlmProvider`	deterministic evidence summary	free, stable tests/demo
`ResilientLlmProvider`	3 retries then fallback	graceful degradation
`ICopilotQueryService`	public answer use case	endpoint thin rakhta hai
`GroundedCopilotQueryService`	guard/retrieve/generate/validate citations	main RAG orchestration
`CopilotTelemetry`	standard ActivitySource	RAG spans vendor-neutral banata hai

Class/interface	Role	Why separate?
`IAiUsageRecorder`	token/cost usage sink	Application ko dashboard vendor se decouple karta hai
`ISupportTool`	named controlled capability	allow-list and policy metadata
`ToolRegistry`	tool lookup	model/user arbitrary code execute nahi kar sakta
`ServiceHealthTool`	mock live health	docs vs live-state distinction
`ConsumerLagTool`	mock Kafka lag	read-only operational evidence
`BookingStatusTool`	synthetic booking lookup	safe domain-relevant demo
`ReplayDeadLetterTool`	approval request only	consequential action boundary
`IApprovalService`	request/decision persistence contract	workflow database decoupling
`SupportWorkflowService`	explicit routing/state trace	agent decisions auditable/deterministic

Infrastructure classes

Class	External responsibility	Failure behavior
`SupportCopilotDbContext`	EF tables, constraints, vector extension	DB exceptions surface to API/logs
`DatabaseInitializer`	local migrate + seed runbooks	startup fails fast if configured DB unavailable
`DocumentIngestionService`	idempotency, upsert, chunk, embed, persist	job error captured; DLQ path can publish
`DeterministicEmbeddingProvider`	offline hashed vectors	always local; not true semantic model
`HttpEmbeddingProvider`	call Python `/embed`	timeout/status/dimension validation
`PostgresHybridRetriever`	tenant-safe lexical+semantic+RRF	unknown tenant returns empty evidence
`OllamaLlmProvider`	local model structured JSON	invalid HTTP/JSON triggers resilient fallback
`KafkaOptions`	broker/topic/group config	environment override friendly
`KafkaIngestionPublisher`	keyed idempotent publication	acks-all/retries; caller sees delivery error
`DirectIngestionPublisher`	no-broker learning path	executes ingestion in request path
`KafkaIngestionWorker`	consume, process, DLQ, manual commit	transient loop retry; poison diagnostic envelope
`ApprovalService`	approval DB operations	missing/expired/already decided rejected
`DependencyInjection`	Infrastructure registrations	one composition method avoids scattered wiring

API classes

Class	Role	Detail
`Program`	composition root	Swagger, JWT, rate limits, endpoints, startup seed
`CorrelationIdMiddleware`	trace header/scope	accepts supplied ID or creates one
`RequestTelemetry`	bounded request + AI metrics	500 item queues, p50/p95, tokens/model/cost snapshot
`RequestTelemetryMiddleware`	measures every request	finally block records failures too
`DemoTokenRequest`	local auth request	default support/operator roles
`ToolExecutionRequest`	JSON tool arguments	tool validates expected properties

Class	Role	Detail
`ApprovalDecisionRequest`	approve/reject input	operator role enforced endpoint-side

Python classes/functions

Symbol	Role
`EmbedRequest`	1-64 input texts validation
`EmbedResponse`	vectors, model and dimensions
`model()`	cached SentenceTransformer load
`embed()`	normalized batch embedding endpoint
`evaluate.py post()`	dependency-free HTTP helper
`evaluate.py main()`	golden dataset metrics/report

Security Reliability and Evaluation

Related: 02 RAG From Zero, 06 System Design Interview, 07 Glossary.

Threat path

```
flowchart LR
    Input --> JWT --> TenantCheck --> PromptGuard --> RetrievalACL --> UntrustedBoundary --> CitationAllowList --> Output
    ToolCall --> RoleCheck --> Approval --> Audit
```

Tenant isolation must happen before LLM context. Output filtering after leakage is too late. Prompt injection is untrusted content attempting to become instruction; system treats retrieved text as data and never gives it policy authority.

Reliability path: correlation ID, bounded retries, timeouts, deterministic fallback, Kafka manual commit, idempotency unique constraint, DLQ, safe abstention and evaluation regression.

Quality path: golden examples -> retrieval ranks -> grounded citations -> safe abstention -> latency/cost -> release threshold. Human review remains necessary for incident advice.

Runbook and Demo

Related: 03 Class and Service Catalog, 06 System Design Interview.

Quick run

```
docker compose up --build
# If your Docker CLI does not support this form:
docker-compose -f compose.yaml up --build
```

Swagger: `http://localhost:5051/swagger`

Token -> Authorize -> hybrid query -> grounded query -> current-health workflow -> DLQ replay workflow -> approval -> injection/tenant negative tests.

Lightweight run

```
docker compose up -d postgres
# Or:
docker-compose -f compose.yaml up -d postgres
dotnet restore ProductionSupportCopilot.sln --configfile NuGet.Config
dotnet run --project src/ProductionSupportCopilot.Api
```

Development mode migrations and demo seed runbooks run when `Database:Initialize=true`.

Database inspection

pgAdmin connection: host `localhost`, port `5432`, database `support_copilot`, user `support`, password `support_dev_password`. Tables live under Schemas -> public -> Tables. View `tenants`, `support_documents`, `document_chunks`, `ingestion_jobs`, `approval_requests`.

Evaluation

```
python evaluation/evaluate.py --base-url http://localhost:5051
```

Common failures

- Port 5432 in use: stop old Postgres container or change compose host port.
- First embedding build slow: PyTorch/model download is large; model cache volume makes later starts faster.
- Swagger 401: obtain token and click Authorize.
- 403: token tenant and request tenant differ, or operator role missing.
- Empty retrieval: seed did not complete; inspect API logs and database.
- Locked DLL on Windows: stop running API/testhost, then build again.
- NuGet NU1900 offline warning: vulnerability feed unreachable; build may still compile from cache, but CI should perform online audit.
- Swagger 404: ensure the API is running on port `5051` and open `/swagger/index.html`.

System Design Interview

Related: 01 Architecture Map, 04 Security Reliability and Evaluation, 05 Runbook and Demo.

Capacity example (assumptions, not measured production facts)

Assume 100 tenants, 10,000 documents each, average 20 chunks = 20 million chunks. A 384D float32 vector is about 1.5 KB raw, roughly 30 GB raw vectors before table/index overhead. If 10 QPS and top-50 candidates, exact scan is no longer acceptable; partition/tenant filters plus HNSW and connection pooling become relevant. Embedding ingestion needs backpressure and autoscaled consumers. LLM concurrency is bounded by provider rate limits and cost budgets.

Interview tradeoffs

- Consistency: document transaction completes before job complete; Kafka gives at-least-once.
- Availability: LLM failure falls back; DB failure prevents grounded answer rather than hallucinating.
- Latency: parallel lexical/vector candidates can reduce time; reranker adds quality and latency.
- Cost: small embedding model, Postgres consolidation, prompt context cap, cached/deduplicated ingestion.
- Security: tenant filter before context, role-gated tools, approval, secrets outside source.
- Scale: start modular monolith; extract ingestion/model gateway on evidence.

Failure questions

If embedding service down: ingestion fails/retries/DLQ; existing indexed docs remain searchable. If LLM down: deterministic cited fallback. If duplicate Kafka message: unique EventId returns duplicate result. If malicious document: trust boundary, no tool authority, citation allow-list, tests. If approval service restarts: persisted request survives; current implementation decision endpoint completes simulated action only, while production needs durable continuation/outbox.

Glossary

Related: 02 RAG From Zero, 04 Security Reliability and Evaluation.

- **LLM:** next-token model capable of language tasks.
- **Embedding:** fixed-size numeric representation of meaning.
- **Vector:** ordered list of numbers.
- **Cosine distance:** angular difference between vectors.
- **RAG:** retrieve external evidence before generation.
- **Chunk:** independently indexed document segment.
- **Hybrid retrieval:** lexical plus semantic search.
- **Reranker:** second-stage candidate ordering model/algorithm.
- **Grounding:** tying claims to provided evidence.
- **Hallucination:** unsupported generated claim.
- **Citation:** source identifier attached to answer claim/evidence.
- **Prompt injection:** untrusted input attempting to override instructions.
- **Tool calling:** model/workflow selecting a controlled application function.
- **Agent:** iterative model-directed workflow; use bounded authority.
- **State machine:** explicit allowed stages/transitions.
- **Idempotency:** retry produces no duplicate effect.
- **DLQ:** dead-letter queue for poison messages.
- **RRF:** reciprocal rank fusion of ranked lists.
- **Recall@K:** fraction where expected result appears in top K.
- **MRR:** mean reciprocal rank; rewards earlier correct result.
- **p95 latency:** 95% requests at or below this duration.
- **Fine-tuning:** changing model weights with task examples.
- **Structured output:** response constrained to schema.
- **SSE:** server-sent events for one-way streaming responses.